Compare basic RNN and LSTM performance for language modelling

To implement and compare SimpleRNN and LSTM models using a common text dataset and evaluate their performance in language modelling.

Algorithm

1. Import TensorFlow, NumPy and the required preprocessing libraries.
2. Define a common text corpus inside the program.
3. Create a tokenizer and convert the text into integer sequences.
4. Generate input sequences and next-word target labels.
5. Pad all sequences to a common length.
6. Divide the data into training and testing portions.
7. Build a SimpleRNN model with an Embedding layer.
8. Build an LSTM model using the same embedding size and output layer.
9. Compile both models using the Adam optimizer and the same loss function.
10. Train both models using identical training data and epochs.
11. Evaluate both models using the same testing dataset.
12. Compare and display their test losses and accuracies.

Program

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
tf.keras.utils.set_random_seed(26)
# Common dataset
corpus = [
"deep learning is useful for image recognition",
"machine learning finds patterns in data",
"recurrent networks process sequential data",
"lstm networks remember long term information",
"simple rnn models process hidden states",
"language models predict the next word",
"neural networks learn features from data",
"deep neural networks contain multiple layers"
] * 10
# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)
seqs = tokenizer.texts_to_sequences(corpus)
# Generate sequences

all_seq = []
for s in seqs:
for i in range(1, len(s)):
all_seq.append(s[:i + 1])
max_len = max(map(len, all_seq))
all_seq = pad_sequences(
all_seq,
maxlen=max_len,
padding='pre'
)
X = all_seq[:, :-1]
y = all_seq[:, -1]
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.2,
random_state=26
)
vocab = len(tokenizer.word_index) + 1
# Function to create models
def build_model(kind):
if kind == "RNN":
recurrent = layers.SimpleRNN(48)
else:
```

```
recurrent = layers.LSTM(48)
model = models.Sequential([
layers.Embedding(vocab, 32),
recurrent,
layers.Dense(vocab, activation='softmax')
])
model.compile(
optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)
return model
# Train and compare
for name in ["RNN", "LSTM"]:

model = build_model(name)
model.fit(
X_train, y_train,
epochs=15,
batch_size=16,
verbose=0
)
loss, accuracy = model.evaluate(
X_test, y_test,
verbose=0
)
print(name)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")
```

Output of Program

```
RNN
Test Loss: Training-dependent value
Test Accuracy: ...
LSTM
Test Loss: Training-dependent value
Test Accuracy: ...
```

Result

Thus, SimpleRNN and LSTM models were implemented and compared using the same text dataset. LSTM uses memory cells and gating mechanisms to capture long-term dependencies, whereas SimpleRNN has a simpler architecture and lower computational complexity.